Bridge

Season: 2 · Structural · Category: GoF · Level: Beginner · Build with Waqar

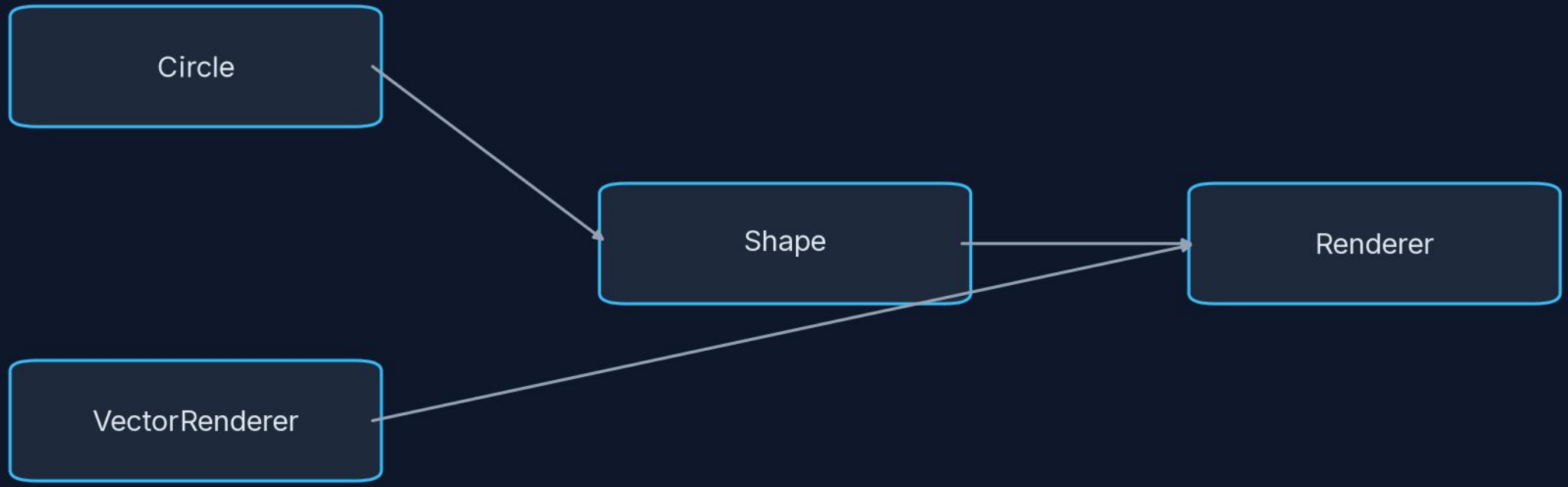
Today's Hook

- Today: Bridge as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Bridge structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Bridge
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/structural/bridge/Shape.java
package com.buildwithwaqar.patterns.gof.structural.bridge;

interface Renderer { String render(String shape); }

record VectorRenderer() implements Renderer {
    public String render(String shape) { return "vector:" + shape; }
}
record RasterRenderer() implements Renderer {
    public String render(String shape) { return "raster:" + shape; }
}

/** Abstraction decoupled from implementation via composition. */
public abstract class Shape {
    protected final Renderer renderer;
    protected Shape(Renderer renderer) { this.renderer = renderer; }
    public abstract String draw();
}

final class Circle extends Shape {
    private final int radius;
    Circle(Renderer renderer, int radius) { super(renderer); this.radius = radius; }
    public String draw() { return renderer.render("circle(" + radius + ")"); }
}
```

Implementation (2)

```
final class Square extends Shape {
    private final int side;
    Square(Renderer renderer, int side) { super(renderer); this.side = side; }
    public String draw() { return renderer.render("square(" + side + ")"); }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.gof.structural.bridge;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Bridge walkthrough ===");
    System.out.println(new Circle(new VectorRenderer(), 3).draw());
    System.out.println(new Square(new RasterRenderer(), 4).draw());
    System.out.println(new Circle(new RasterRenderer(), 2).draw());
  }
}
```

Walkthrough Output

```
=== Bridge walkthrough ===
vector:circle(3)
raster:square(4)
raster:circle(2)
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Bridge
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Bridge → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.